JSP

JSP stands for **java server pages** and it is used for develop the dynamic web page and it is server side scripting language.

JSP technology is used to create web application just like Servlet technology. It can be thought of as an extension to Servlet because it provides more functionality than servlet such as expression language, JSTL (The Jakarta Standard Tag Library (JSTL; formerly JavaServer Pages Standard Tag Library) is a component of the Java EE Web application development platform.), etc.

A JSP page consists of HTML tags and JSP tags. The JSP pages are easier to maintain than Servlet because we can separate designing and development. It provides some additional features such as Expression Language, Custom Tags, etc.

Q. Why use JSP if we have servlet for dynamic web pages?

---

1. JSP provide tags to us for generate the dynamic web content
2. Not need to write complicated code for generates dynamic web pages.
3. Not need to complicated configuration like as servlet
4. Not need to create objects manually like as session, cookie, ServletContext, ServletConfig, etc.
JSP provide some inbuilt object to us
5. Easy to integrate UI in dynamic web pages as compare with servlet
Etc.

Note: Every JSP page internally works as servlet page because when we run JSP page then internally JSP page convert in servlet by JSP container and servlet get compiled and executed by tomcat.

Q. What is JSP container?

JSP container is internal API present in web server which helps us to convert jsp to servlet

---

The Lifecycle of a JSP Page

The JSP pages follow these phases:

- Translation of JSP Page
- Compilation of JSP Page
- Classloading (the classloader loads class file)

- Instantiation (Object of the Generated Servlet is created).
- Initialization (the container invokes jspInit() method).
- Request processing (the container invokes _jspService() method).
- Destroy (the container invokes jspDestroy() method).

Note: jspInit(), _jspService() and jspDestroy() are the life cycle methods of JSP.

JSP API

The JSP API consists of two packages:

1. javax.servlet.jsp
2. javax.servlet.jsp.tagext

The javax.servlet.jsp package has two interfaces and classes. The two interfaces are as follows:

1. JspPage
2. HttpJspPage

The classes are as follows:

1. JspWriter
2. PageContext
3. JspFactory
4. JspEngineInfo
5. JspException
6. JspError

The JspPage interface

According to the JSP specification, all the generated servlet classes must implement the JspPage interface. It extends the Servlet interface. It provides two life cycle methods.

Methods of JspPage interface

1. **public void jspInit():** It is invoked only once during the life cycle of the JSP when JSP page is requested firstly. It is used to perform initialization. It is same as the init() method of Servlet interface.
2. **public void jspDestroy():** It is invoked only once during the life cycle of the JSP before the JSP page is destroyed. It can be used to perform some clean-up operation.

The HttpJspPage interface

The HttpJspPage interface provides the one life cycle method of JSP. It extends the JspPage interface.

Method of HttpJspPage interface:

1. **public void _jspService():** It is invoked each time when request for the JSP page comes to the container. It is used to process the request. The underscore _ signifies that you cannot override this method.

How to work with JSP

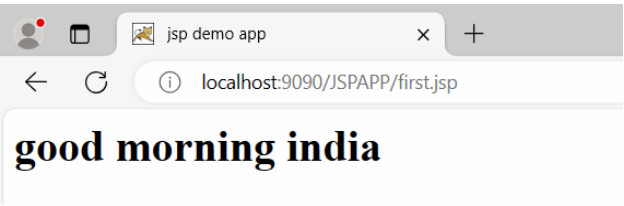
If you want to work with JSP we have following steps.

1. Create Project using standard folder structure of apache
Note: refer servlet folder structure
2. Create HTML page and save it using .jsp in root folder of project under web app
first.jsp

```
<html>
 <head>
  <title>jsp demo app</title>
 </head>
<body>
  <h1>good morning india</h1>
</body>
</html>
```

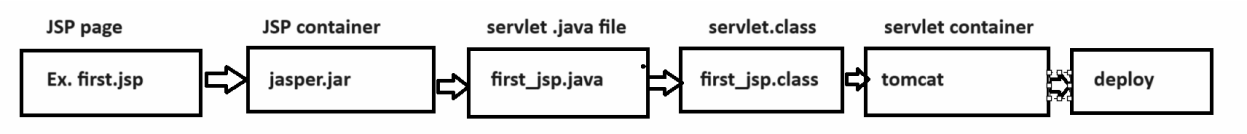
3. Execute or deploy or run the jsp page on tomcat

Syntax: http://localhost:portnumber/projectname/urlname



Note: After executing the JSP page it is internally convert in servlet.

Process of JSP to Servlet Conversion



When your JSP convert in servlet then internally we get three methods which work as life cycle method of JSP

_jspInit(),
_jspService(final ServletRequest request, ServletResponse response)
_jspDestroy()

If we want to work with JSP we have some inbuilt tags provided by JSP to us and some inbuilt objects provided by JSP to us

JSP Scripting elements

The scripting elements provide the ability to insert java code inside the jsp. We have the following types of scripting elements:

Tags of JSP

Declaration: declaration tag in JSP is used for declare variable in jsp, or objects in jsp page etc
Syntax:

```
<%!
    perform declaration here
 %>
```

Example:

```
<%!
  int a,b,c;
 %>
```

Scriptlet: Scriptlet tag is used for writing logics in jsp page like as for loop, if or any other logics

Syntax:

```
<%   write here your logics
 %>
```

Example:

```
<%
 a=100;
 b=200;
 c=a+b;
%>
```

Expression: expression tag is used for display the output page on web page.

Syntax:

<%= expression %>

Example: <%=c%>

it is like as <% out.println(c); %>

Note: we cannot use semi colon in expression tag

<%=c;%>

If you give semi colon in expression tag it is consider <% out.println(c;);%>

Directive: directive tag is used for add the page specified attribute on JSP page, add external tag library in JSP page or include another page content in JSP page.

Syntax: <%@attribute subattribute=”parameter” %>

There are three types of attribute of JSP page

1) Page 2) include 3) taglib

Note: we will discuss these attribute later in this chapter.

Objects of JSP

---

out: this is inbuilt object of JspWriter class and it is child of PrintWriter internally and it is used for display the output on web page.

request: it is internally member of HttpServletRequest means we can use all methods of HttpServletRequest using request object.

The **JSP request** is an implicit object of type HttpServletRequest i.e. created for each jsp request by the web container. It can be used to get request information such as parameter, header information, remote address, server name, server port, content type, character encoding etc.

response: it is internally object of HttpServletResponse

In JSP, response is an implicit object of type HttpServletResponse. The instance of HttpServletResponse is created by the web container for each jsp request.

It can be used to add or manipulate response such as redirect response to another resource, send error etc.

session: it is object of HttpSession interface

In JSP, session is an implicit object of type HttpSession. The Java developer can use this object to set, get or remove attribute or to get session information.

config: it is object of ServletConfig

In JSP, config is an implicit object of type *ServletConfig*. This object can be used to get initialization parameter for a particular JSP page. The config object is created by the web container for each jsp page.

page : page points to this member current working page object

In JSP, page is an implicit object of type Object class. This object is assigned to the reference of auto generated servlet class. It is written as:

Object page=this;

For using this object it must be cast to Servlet type. For example:

<% (HttpServlet)page.log("message"); %>

Since, it is of type Object it is less used because you can use this object directly in jsp.
For example:

<% this.log("message"); %>

context: it is object of PageContext

In JSP, pageContext is an implicit object of type PageContext class. The pageContext object can be used to set, get or remove attribute from one of the following scopes:

- page
- request
- session
- application

In JSP, page scope is the default scope.

application: it is object of ServletContext

In JSP, application is an implicit object of type *ServletContext*.

The instance of ServletContext is created only once by the web container when application or project is deployed on the server.

This object can be used to get initialization parameter from configuaration file (web.xml). It can also be used to get, set or remove attribute from the application scope

exception : it is inbuilt object of Exception
In JSP, exception is an implicit object of type java.lang.Throwable class. This object can be used to print the exception. But it can only be used in error pages. It is better to learn it after page directive

Note: these object we can use only within a service method means within scriptlet tag

Create JSP project using eclipse and declare two variables and calculate its addition and display it.
Steps.
1. Create Dynamic web project
2. Create JSP page under webapp folder

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<!-- declaration tag -->
<%!
   int a,b,c;
%>
<%
 a=100;
 b=200;
 c=a+b;
 out.println("Addition is "+c);
%>
</body>
</html>
```

Output:

if we think about above code we calculate addition of fix values so we want to accept input from keyboard and then calculate addition.

so we design HTML form and provide input in form and submit to server then calculate addition

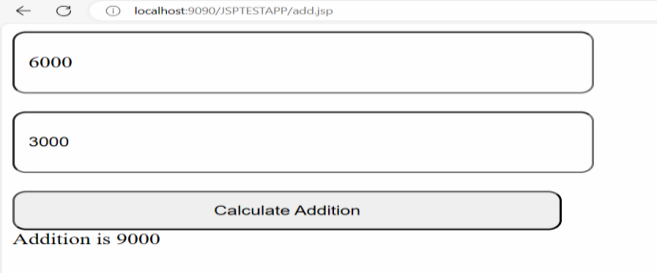
Source

add.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
<style>
 input{
   width:400px;
   height:40px;
   border-radius:10px;
   padding:10px;
 }
</style>
</head>
<body>
<!-- declaration tag -->
<%!
  int a,b,c;
%>
<form name='frm' action='' method='POST'>
<input type='text' name='first' value=''/><br/><br/>
<input type='text' name='second' value=''/><br/><br/>
<input type='submit' name='s' value='Calculate Addition'/>
</form>
<%
String btn=request.getParameter("s");
if(btn!=null){
 a=Integer.parseInt(request.getParameter("first"));
 b=Integer.parseInt(request.getParameter("second"));
 c=a+b;
```

```
 out.println("Addition is "+c);
}
%>
</body>
</html>
```

Output:



Example: WAP to design HTML page with single text box and button and input number in textbox and print its table.

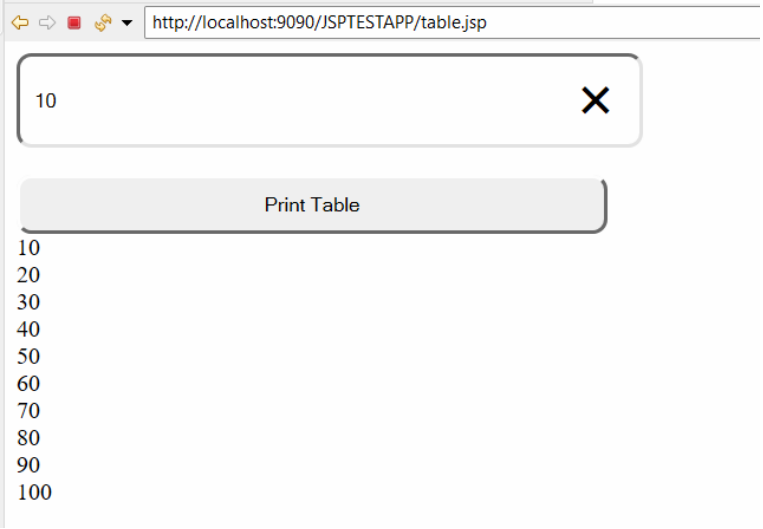


table.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
<style>
  input{
    width:400px;
    height:40px;
```

```
   border-radius:10px;
   padding:10px;
  }
</style>
</head>
<body>
<%!
 int no, tab;
%>
<form name='frm' action='' method='POST'>
 <input type='text' name='num' value=''/><br/><br/>
 <input type='submit' name='s' value='Print Table'/>
 </form>
 <%
   String btn=request.getParameter("s");
   if(btn!=null){
          no=Integer.parseInt(request.getParameter("num"));
          for(int i=1;i<=10;i++){
                 tab=i*no;
                 out.println(tab+"<br>");
          }
   }

 %>
</body>
</html>
```

If we think about above code we print the table but we want to display table like as

Iteration	X	Number	=	Result
1	X	5	=	5
2	X	5	=	10
3	X	5	=	15
4	X	5	=	20

Note: if we want to show the output like as we need to generate dynamic HTML using JSP

How to generate dynamic HTML using JSP

If we want to generate dynamic HTML using JSP we have two ways

1. Generate using out.println () tag

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
   pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
<style>
 input{
   width:400px;
   height:40px;
   border-radius:10px;
   padding:10px;
 }
</style>
</head>
<body>
<%!
 int no, tab;
%>
<form name='frm' action='' method='POST'>
 <input type='text' name='num' value=''/><br/><br/>
 <input type='submit' name='s' value='Print Table'/>
 </form>
 <%
   String btn=request.getParameter("s");
   if(btn!=null){
          no=Integer.parseInt(request.getParameter("num"));
   out.println("<table border='5' width='80%'>");
   out.println("<tr><th>Iteration</th><th>X</th><th>Number</th><th>=</th><th>Result</th></tr>");
          for(int i=1;i<=10;i++){
                 tab=i*no;
    out.println("<tr>");
    out.println("<td align='center'>"+i+"</td>");
    out.println("<td align='center'>X</td>");
    out.println("<td align='center'>"+no+"</td>");
    out.println("<td align='center'>=</td>");
    out.println("<td align='center'>"+(no*i)+"</td>");
    out.println("</tr>");
          }
   out.println("</table>");
   }
```

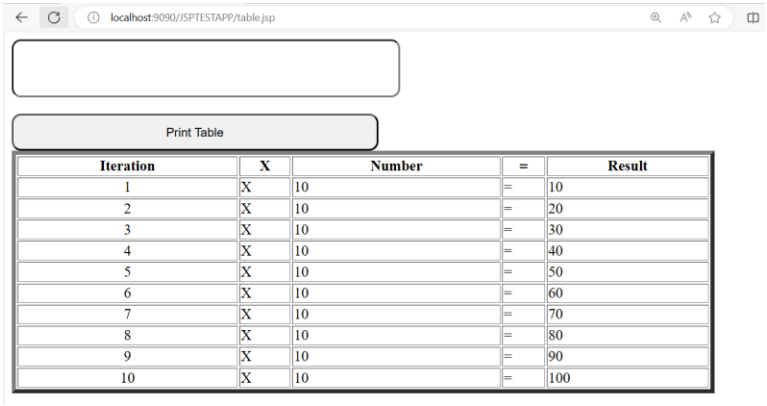
```
 %>
</body>
</html>
```

2. Generate using Tag breaking Technique.

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
	pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
<style>
input {
	width: 400px;
	height: 40px;
	border-radius: 10px;
	padding: 10px;
}
</style>
</head>
<body>
	<%!int no, tab;%>
	<form name='frm' action='' method='POST'>
		<input type='text' name='num' value='' /><br />
		<br /> <input type='submit' name='s' value='Print Table' />
	</form>
	<%
	String btn = request.getParameter("s");
	if (btn != null) {
		no = Integer.parseInt(request.getParameter("num"));
	%>
	<table border='5' width='80%'>
		<tr>
			<th>Iteration</th>
			<th>X</th>
			<th>Number</th>
			<th>=</th>
			<th>Result</th>
		</tr>
		<%
		for (int i = 1; i <= 10; i++) {
```

```
                tab = i * no;
            %>
            <tr>
                <td align='center' ><%=i%></td>
                <td>X</td>
                <td><%=no%></td>
                <td>=</td>
                <td><%=tab%></td>
            </tr>
            <%
            }
            %>
        </table>
        <%
        }
        %>
</body>
</html>
```

Output



Iteration	X	Number	=	Result
1	X	10	=	10
2	X	10	=	20
3	X	10	=	30
4	X	10	=	40
5	X	10	=	50
6	X	10	=	60
7	X	10	=	70
8	X	10	=	80
9	X	10	=	90
10	X	10	=	100

Directive Tags and its attribute

Directive tags is used for apply page level attribute on jsp page as well as include another page content on jsp page and use the third party tag libraries in jsp

The **jsp directives** are messages that tell the web container how to translate a JSP page into the corresponding servlet.

There are three types of attribute of Directives tag

---

1. page: Page directives is used for add the page level attributes on JSP page.

Syntax: <%@page attribute=”values” %>

Attribute of JSP page

---

import: this is sub attribute of page attribute of JSP page and which is used for import package in application**.**

<%@page import=”packagename, packagename” %>

Example: we want to create web page which is used for display system date on web page

Source code

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
   pageEncoding="ISO-8859-1"%>
<%@page import="java.util.*" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<%!
  Date d;
%>

<%
 d = new Date();
%>
<h1>Today Date is <%=d%></h1>
</body>
</html>
```

Output:

contentType: this attribute is used for decide the response type from server to client
by default it is text/html

Syntax: <%@ page language="java" contentType="text/html” %>

isErrorPage : isErrorPage is sub attribute of page and this page decide current page may be contain exception or not or may be generate run time exception or not if we set true the meaning is your current page may be generate exception at run time if we set false then current page not generate exception at run time.

errorPage: errorPage attribute set error page or target errorPage means when your working page generate exception at run time then server call error page
note: isErrorPage and errorPage attribute use at same time**.**

Example: we want to create application input two values and calculate its division
if think about division then there is possibility of exception if user input second value as 0 then JVM may be generate run time exception

Example:
div.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
   pageEncoding="ISO-8859-1"  isErrorPage="true" errorPage="error.jsp" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
<style>
 input{
   width:400px;
   height:40px;
   border-radius:10px;
   padding:10px;
 }
</style>
</head>
<body>
<%!
 int a,b,c;
 %>
<form name='frm' action='' method='POST'>
<input type='text' name='first' value=''/><br/><br/>
<input type='second' name='second' value=''/><br/><br/>
<input type='submit' name='s' value='Calculate Division'/>
</form>
<%
   String btn=request.getParameter("s");
   if(btn!=null){
         a=Integer.parseInt(request.getParameter("first"));
         b=Integer.parseInt(request.getParameter("second"));
         c=a/b;
         out.println("<h1>Division is "+c+"</h1>");
```

```
   }
%>
</body>
</html>
```

error.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
   pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
 <h1>Second value should not zero</h1>
</body>
</html>
```

session: this attribute decide session object can use on web page or not if we set false then we cannot use session object on web page and if set true then we can use session object on web page.

Syntax: <%@page session=”true | false” %>

Example:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
   pageEncoding="ISO-8859-1"%>
<%@page session="false" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
 <h1>Second value should not zero</h1>
 <%
  session.setAttribute("u", "Ramesh");
 %>
</body>
</html>
```

Note: if we think about above code we have statement session.setAttribute(“u”, ”ramesh”);
will generate error to us because we set session=”false” in directives tag means we disable session from this web page

charSet : this method is used for set uni code character set on web page like as
Syntax: <%@page charset=”charset” %>

Example:<%@ page language=*"java"* contentType=*"text/html; charset=ISO-8859-1" %>*

bufferSize : this attribute set the buffer size of web page
Syntax: <%@page bufferSize=”17kb” %>

isELIgnored: this attribute is used for decide expression language should use on web page or not
if we set false then we can use expression language on web page if we set true then we cannot use expression language on web page.

Syntax: <%@page isELIngored=”true | false” %>

language: language attribute decide language with jsp page.
Example: <%@page language=”java” %>

2. include: this attribute help us to include the another page content in JSP.
Syntax: <%@include file=”pagename” %>

Example: we want to use the common navigation bar in all jsp pages.

nav.jsp.

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
   pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
<style>
  ul{
    list-style-type:none;
    width:100%;
    background-color:black;
    padding:10px;
  }
```

```
  ul li{
    display:inline;
    padding:10px;
  }
  ul li a {
    text-decoration:none;
    color:white;
  }
</style>
</head>
<body>
 <ul>
   <li><a href='add.jsp'>Addition</a></li>
    <li><a href='div.jsp'>Division</a></li>
  </ul>
</body>
</html>
```

add.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
   pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
<style>
 input{
   width:400px;
   height:40px;
   border-radius:10px;
   padding:10px;
 }
</style>
</head>
<body>
<!-- declaration tag -->
<%!
  int a,b,c;
%>
<%@include file="nav.jsp" %>
<br/>
```

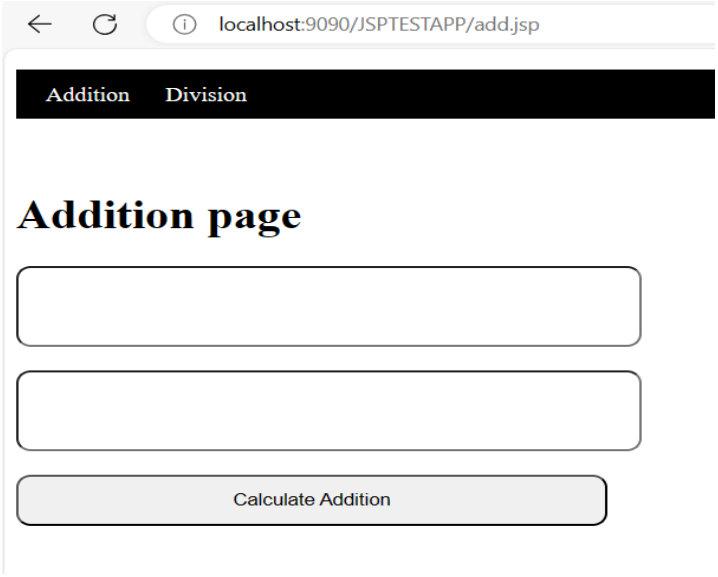
```
<h1>Addition page</h1>
<form name='frm' action='' method='POST'>
<input type='text' name='first' value=''/><br/><br/>
<input type='second' name='second' value=''/><br/><br/>
<input type='submit' name='s' value='Calculate Addition'/>
</form>
<%
String btn=request.getParameter("s");
if(btn!=null){
 a=Integer.parseInt(request.getParameter("first"));
 b=Integer.parseInt(request.getParameter("second"));
 c=a+b;
 out.println("Addition is "+c);
}
%>

</body>
</html>
```

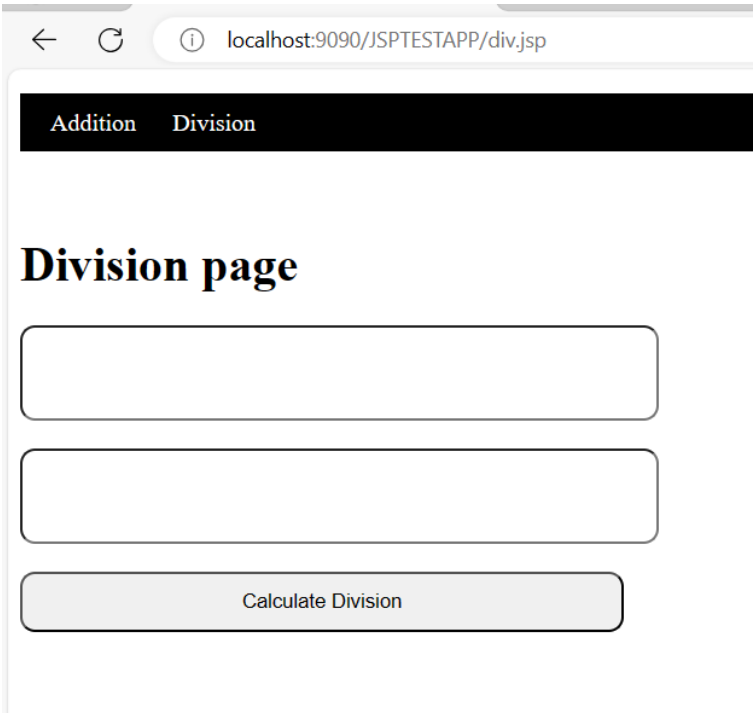
div.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
   pageEncoding="ISO-8859-1"  isErrorPage="true" errorPage="error.jsp" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
<style>
 input{
   width:400px;
   height:40px;
   border-radius:10px;
   padding:10px;
 }
</style>
</head>
<body>
<%!
 int a,b,c;
 %>
 <%@include file="nav.jsp" %>
<br/>
 <h1>Division page</h1>
```

```
<form name='frm' action='' method='POST'>
<input type='text' name='first' value=''/><br/><br/>
<input type='second' name='second' value=''/><br/><br/>
<input type='submit' name='s' value='Calculate Division'/>
</form>
<%
   String btn=request.getParameter("s");
   if(btn!=null){
         a=Integer.parseInt(request.getParameter("first"));
         b=Integer.parseInt(request.getParameter("second"));
         c=a/b;
         out.println("<h1>Division is "+c+"</h1>");
   }
%>
</body>
</html>
```



div.jsp



3. taglib: taglib directives is used for use third party tag library in jsp page like as JSTL, spring mvc form tag library etc

Syntax:

<%@taglib uri=”uri of page” prefix=”starting letter of tag” %>

uri: this attribute decide the uri of web page

prefix: this attribute decide starting letter of tag for use tag from tag library